

# **UNIT-3 Javascript**

## **Dynamic HTML (DHTML)**

Dynamic HTML refers to a combination of technologies that allow web pages to be interactive and dynamically updated without requiring a full page reload. It encompasses HTML, CSS, and JavaScript, enabling elements on a web page to change in response to user actions or other events. DHTML techniques include animations, drag-and-drop functionality, and real-time updates of content. It enhances user experience by creating more responsive and engaging web applications compared to static HTML pages.

A scripting language is a type of programming language designed for automating specific tasks. Unlike compiled languages, scripts are typically executed directly by an interpreter, without the need for compilation. Scripting languages are commonly used for tasks such as web development, system administration, and automation. One example of a scripting language is Javascript, known for its simplicity, versatility, and extensive libraries.

**JavaScript** is a lightweight, cross-platform and interpreted compiled programming language. It is also known as the scripting language for webpages.

It is well-known for the development of web pages, and many non-browser environments also use it.

## **Features of JavaScript**

- Earlier websites were mostly static, after JS was created dynamic Web sites were made.
- Functions in javascript are objects. They may have properties and methods just like other objects. They can be passed as arguments in other functions.
- Can handle date and time.
- No compiler is needed.
- Performs Form Validation although the forms are created using HTML.

## **Benefits of JavaScript**

- It is widely supported in Web browsers;
- It gives easy access to the document objects and can manipulate most of them;

- JavaScript can give interesting animations without the long download times associated with many multimedia data types;
- Web surfers don't need a special plug-in to use your scripts.
- JavaScript is relatively secure.

## **Problems with JavaScript**

- Most scripts rely upon manipulating the elements of the DOM. Support for a standard set of objects currently doesn't exist and access to objects differs from browser to browser;
- If your script doesn't work then your page is useless.
- Because of the problems of broken scripts many Web surfers disable JavaScript support in their browser;
- Scripts can run slowly and complex scripts can take a long time to start up.

## **VARIABLES**

Variables give us the ability to manipulate data through reference instead of actual value. Variables are names assigned to values. Variables are containers that hold values.

### **Variable Names**

There are strict rules governing how variables are named in JavaScript:

- Names must begin with a letter, digit, or underscore (\_).
- Spaces are not allowed in names.
- Names are case-sensitive
- Reserved words, which are part of the language itself, cannot be used as variable names.

Example: `visitor_name = visitor_name + 45.32`

### **Creating a variables**

In JavaScript, variables are declared using the keyword `var` followed by the variable name. Their data type doesn't need to be specified during declaration, unlike in languages such as Pascal and C++. Values can be assigned to variables later in the code, or variables can be left uninitialized. It's possible to copy the value of one variable directly into another during declaration.

In Javascript for creating variables use “var” keyword

```
var first;                // Declaration
var second = 23;          // Assignment or Initialization
```

Example: Using Variables in JavaScript

```
<html>
<head><title>A Sample JavaScript</title></head>
<body >
<script language="javascript">
var visitor_name;
visitor__name = prompt("Enter Your Name", " ");
document. writeln("<hl>Your name is</hl>");
document.writeln("<h2>" + visitor__name + "</h2>") ;
document. close ();
</script>
</body>
</html>
```

## Data Types

JavaScript is a dynamic type language, means you don't need to specify type of the variable because it is dynamically used by JavaScript engine.

JavaScript has only four types of data:

- **Numeric:** These are basic numbers. They can be integers such as 2, 22, and 2,222,000 or floating point values like 23.42, -56.01, and 2E45.
- **Strings:** These are collections of characters that are not numbers. The value of a string can even contain spaces and may be totally made from digits.

Example: visitor\_name = "Chris Bates"

- **Boolean:** variables hold the values true and false. These are used a lot in programming to hold the result of conditional tests.
- **Null:** This is used when you don't yet know something. A null value means one that has not yet been decided.

## Operators

JavaScript operators are symbols used to operate the operands. Operators are used to perform specific mathematical and logical computations on operands.

Table 5.2: JavaScript Operators

| Operator | Meaning                                                                                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| +        | If the arguments are numbers then they are added together. If the arguments are strings then they are concatenated <sup>7</sup> and the result returned. |
| -        | If supplied with two operands this subtracts one from the other. If supplied with a single operand it reverses its sign.                                 |
| *        | Multiplies two numbers together.                                                                                                                         |
| /        | Divides the first number by the second.                                                                                                                  |
| %        | Modulus Division returns the integer remainder from a division.                                                                                          |
| !        | Logical NOT returns false if the operand evaluates to true. Otherwise it returns true.                                                                   |
| >        | Greater than returns true if the left operand is greater than the right.                                                                                 |
| >=       | Returns true if the left operand is greater than or equal to the right one.                                                                              |
| <        | Returns true if the left operand is less than the right.                                                                                                 |
| ==       | Returns true if the two operands are equal.                                                                                                              |
| <=       | Returns true if the left operand is less than or equal to the right one.                                                                                 |
| !=       | Returns true if the two operands are not equal.                                                                                                          |
| &&       | Logical AND returns true if both operands are true. Otherwise returns false.                                                                             |
|          | Logical OR returns true if one or both operands are true, otherwise returns false.                                                                       |
| =        | Assigns a value to a variable                                                                                                                            |
| +=       | Adds two numbers then assigns the result to the one on the left of the expression.                                                                       |
| -=       | Subtracts the term on the right from the term on the left, then assigns the result to the one on the left of the expression.                             |
| *=       | Multiplies two values then assigns the result to the one on the left of the expression.                                                                  |
| /=       | Divides the term on the left by the term on the right and then assigns the result to the one on the left of the expression.                              |
| %=       | Performs modulus division then assigns the result to the one on the left of the expression.                                                              |
| ++       | Auto-increment, increases the value of its (integer) argument by one.                                                                                    |
| --       | Auto-decrement, decreases the value of an integer by one.                                                                                                |

## Logical Operators

| Operator | Description | Example                   |
|----------|-------------|---------------------------|
| &&       | and         | (x < 10 && y > 1) is true |
|          | or          | (x==5    y==5) is false   |
| !        | not         | !(x==y) is true           |

## Arithmetic Operators

| Operator | Description                     | Example  | Result of x | Result of y |
|----------|---------------------------------|----------|-------------|-------------|
| +        | Addition                        | $x=y+2$  | 7           | 5           |
| -        | Subtraction                     | $x=y-2$  | 3           | 5           |
| *        | Multiplication                  | $x=y*2$  | 10          | 5           |
| /        | Division                        | $x=y/2$  | 2.5         | 5           |
| %        | Modulus<br>(division remainder) | $x=y\%2$ | 1           | 5           |
| ++       | Increment                       | $x=++y$  | 6           | 6           |
| --       | Decrement                       | $x=--y$  | 4           | 4           |

## Assignment Operators

| Operator | Example | Same As  | Result |
|----------|---------|----------|--------|
| =        | $x=y$   |          | $x=5$  |
| +=       | $x+=y$  | $x=x+y$  | $x=15$ |
| -=       | $x-=y$  | $x=x-y$  | $x=5$  |
| *=       | $x*=y$  | $x=x*y$  | $x=50$ |
| /=       | $x/=y$  | $x=x/y$  | $x=2$  |
| %=       | $x\%=y$ | $x=x\%y$ | $x=0$  |

## Comparison Operators

| Operator | Description                          | Comparing        | Returns      |
|----------|--------------------------------------|------------------|--------------|
| ==       | is equal to                          | $x==8$           | <i>false</i> |
|          |                                      | $x==5$           | <i>true</i>  |
| ===      | is exactly equal to (value and type) | $x===\text{"5"}$ | <i>false</i> |
|          |                                      | $x===5$          | <i>true</i>  |
| !=       | is not equal                         | $x!=8$           | <i>true</i>  |
| !==      | is not equal (neither value or type) | $x!==\text{"5"}$ | <i>true</i>  |
|          |                                      | $x!==5$          | <i>false</i> |
| >        | is greater than                      | $x>8$            | <i>false</i> |
| <        | is less than                         | $x<8$            | <i>true</i>  |
| >=       | is greater than or equal to          | $x>=8$           | <i>false</i> |
| <=       | is less than or equal to             | $x<=8$           | <i>true</i>  |

## Typeof Operator

The typeof operator in JavaScript is used to determine the type of a variable or an expression. It returns a string indicating the type of the operand. It's particularly useful when you need to dynamically check the type of a variable or when you want to handle different types of data differently in your code.

Example:

```
var x = 42; // Declare and initialize a variable
var y = "Hello"; // Another variable with a string value
var z = true; // Variable with a boolean value
```

```
console.log(typeof x); // Output: "number"
console.log(typeof y); // Output: "string"
console.log(typeof z); // Output: "boolean"
```

## prompt()

The prompt() function in JavaScript is used to display a dialog box with a message prompting the user to input text. This function is often used to interactively gather input from the user in web applications.

Example:

```
var userInput = prompt("Please enter your name:");
console.log("User entered:", userInput);
```

## STRING MANIPULATION

Strings: These are collections of characters that are not numbers. The value of a string can even contain spaces and may be totally made from digits.

Example: var visitor\_name = "Chris Bates"

### 1)charAt(index)

This function returns the character which is at position index in the string.

**Example:** var str="COLLEGE";  
var ch=str.charAt(1); // Output: 'O'

## 2)concat("string"[, "string"[,..."string"]])

This method adds two or more strings and returns a new string.

When you call concat() on a string variable, you can pass one or more strings as arguments, and it will merge them with the original string.

### Example:

```
var you = "Ram";  
var yourage = "25";  
var result = you.concat(yourage);  
console.log(result);    // Output: "Ram25"
```

## 3)indexOf ("search"[, offset])

The string is searched for the string or quoted character in the first parameter. If the search is successful, the index of the start of the target string is returned. If the search is unsuccessful the operation returns -1.

### Example: <script>

```
    var sentence = "Hello, world!";  
    var index = sentence.indexOf("world");  
    if (index !== -1) {  
        console.log("The word 'world' starts at index: " + index);  
    } else {  
        console.log("The word 'world' was not found.");  
    }  
</script>                                //Output: The word 'world' starts at index: 7
```

## 4)The lastIndexOf()

This function in JavaScript is similar to the indexOf() function, but it searches for a substring starting from the end of the string and moving towards the beginning.

It returns the index of the last occurrence of the substring within the string, or -1 if the substring is not found.

### Example:

```
var sentence = "Hello, world!";  
var lastIndex = sentence.lastIndexOf("o");  
console.log("Last index of 'o': " + lastIndex);    //Output: Last index of 'o': 8
```

## 5)Length

length Value which holds the number of characters in the string.

This is not a function and so don't have to place parentheses when using it.

**Example:**

```
var str = "Hello, world!";  
var lengthOfStr = str.length;  
console.log(lengthOfStr);           // Output: 13
```

**6) split(separator[, limit])**

The split () function breaks the string apart whenever it encounters the character passed in as the first parameter.

The pieces of the string are stored in an array.

split () has an optional second parameter which is an integer value indicating how many of the pieces are to be stored in the array.

**Example:**

```
var agent = "Javascript Split Function";  
var words = agent.split(" ");  
for (var i = 0; i < words.length; i++) {  
    document.writeln("<p>" + words[i] + "</p>");  
}
```

Output: Javascript  
Split  
Function

**7) substr(index[,length])**

This function returns a substring which starts at the character indicated by the index parameter. The substring continues either to the end of the string or for the number of characters indicated by the length parameter.

If the index is greater than the length of the string then nothing is returned.

If it is negative then it is taken as the offset from the end of the string working backwards along its length.

If length is 0, or a negative number is provided then no characters are returned.

**Example:**

```
var agent = "Javascript Split Function";  
var piece = agent.substr(4, 16);  
document.writeln("<p>" + piece + "</p>");
```

//Output: script Split Fun

**8) substring(index1[, index2])**



Returns the set of characters which starts at index1 and continues up to, but does not include, the character at index2.

The following rules apply:

- 1 If index1 is less than 0, it will be treated as 0.
2. If index2 is greater than the length of the string, it is treated as the length of the string.
3. If the two index values are equal, an empty string is returned.
4. If index2 is missing, all characters up to the end of the string are taken.
5. If index1 is greater than index2, a runtime error occurs.

**Example:**

```
var agent = "Javascript Split Function";  
    var piece = agent.substring(7, 12);  
    document.writeln("<p>" + piece + "</p>");    //Output: Split
```

## **9)toLowerCase()**

Converts all characters in the string to lower case.

## **10)toUpperCase()**

Converts all characters in the string to upper case.

**Example:**

```
var str1 = "Hello, World!";  
    var lowercaseStr = str1.toLowerCase();  
    console.log(lowercaseStr);                // Output: "hello, world!"  
  
var str2 = "hello, world!";  
    var uppercaseStr = str2.toUpperCase();  
    console.log(uppercaseStr);                // Output: "HELLO, WORLD!"
```

## **MATHEMATICAL FUNCTIONS**

The Javascript Math object allows you to perform mathematical tasks on numbers.

Mathematical functions and values are part of a builtin JavaScript object called Math. All functions and attributes used in complex mathematics must be accessed via this object.

This is usually done by preceding the function name with the object name:

```
var area = Math.PI * (r * r);  
var next = Math.ceil(area);
```

### 1)NaN

The term "NaN" stands for "Not a Number" in JavaScript. It is a special value that indicates a value is not a valid number.

Example: `var result = "Hello" / 2;`

`// result will be NaN because "Hello" cannot be divided by 2.`

### 2) abs (value)

Returns the absolute value the number passed into it.

Example: `abs(-5)` returns 5, as the absolute value of -5 is 5.

### 3)acos (value), asin (value), atan (value)

These functions return the arccosine, arcsine and arctangent, respectively, of the value passed into them.

All return values are in radians.

#### **acos(value):**

Example: `acos(0.5)` returns 1.0471975511965976, which is approximately equal to 60 degrees in radians. This represents the arccosine of 0.5.

#### **asin(value):**

Example: `asin(0.5)` returns 0.5235987755982988, which is approximately equal to 30 degrees in radians. This represents the arcsine of 0.5.

#### **atan(value):**

Example: `atan(1)` returns 0.7853981633974483, which is approximately equal to 45 degrees in radians. This represents the arctangent of 1.

### 4)atan2(valuel, value2)

Returns the arctangent, in radians, of the quotient of the values passed into it.

Example: `atan2(1, 2)` returns 0.4636476090008061, which is approximately equal to 26.57 degrees in radians. This represents the arctangent of the quotient of 1 and 2.

### 5)ceil (value)

Returns the smallest integer which is greater than, or equal to, the value passed in.

Example: `ceil(3.14)` returns 4, as the smallest integer greater than or equal to 3.14 is 4.

### **6) floor (value)**

Returns the largest integer which is smaller than, or equal to, the number passed in.

Example: `floor(3.14)` returns 3, as the largest integer smaller than or equal to 3.14 is 3.

### **7) cos (value), sin (value), tan (value)**

These return the cosine, sin and tangent, respectively, of the value passed in as an argument.

Example: `cos(0)` returns 1, as the cosine of 0 radians is 1.

Example: `sin( $\pi/6$ )` returns 0.5, as the sine of  $\pi/6$  radians (30 degrees) is 0.5.

Example: `tan( $\pi/4$ )` returns 1, as the tangent of  $\pi/4$  radians (45 degrees) is 1.

### **8)isNan (value)**

This function returns true if its argument is not a number and false if it is numeric.

Example 1: `isNan(5)` returns false, as 5 is a numeric value.

Example 2: `isNan("Hello")` returns true, as "Hello" is not a numeric value.

### **9)log (value)**

Returns the natural logarithm of its argument. If the argument is not a number or is a negative number then NaN will be returned.

Example 1: `log(10)` returns 2.302585092994046, which is the natural logarithm of 10.

Example 2: `log(-5)` returns NaN, as the natural logarithm of a negative number is undefined.

### **10)max(valuel, value2)**

Returns the larger of its arguments.

Example: `max(10, 5)` returns 10, as 10 is the larger of the two values.

### **11)min(valuel, value2)**

Returns the smaller of its arguments.

Example: `min(10, 5)` returns 5, as 5 is the smaller of the two values.

### **12)The parseFloat()**

This function in JavaScript takes a string as input and attempts to parse it into a floating-point number.

It scans the string from the beginning, recognizing numeric digits (0-9), signs (+ and -), decimal points, and exponents.

If the string contains any non-numeric characters or begins with an invalid character, `parseFloat()` returns NaN, indicating the parsing was unsuccessful.

Example:

```
let num = parseFloat("3.14");  
console.log(num); // Output: 3.14  
let invalidNum = parseFloat("Hello");  
console.log(invalidNum); // Output: NaN
```

### **13)The parseInt()**

This function in JavaScript is used to parse a string and extract an integer value from it. It scans the string from left to right until it encounters a character that is not a valid digit in the specified radix or base.

At that point, it stops parsing and returns the integer value it has found so far.

If the first character of the string is not a valid digit in the specified radix, `parseInt()` returns NaN, indicating that parsing was unsuccessful.

The set of valid characters depends upon the supplied radix:

- in base 2 only the characters 0 and 1 are permitted,
- in base 10 the digits 0 through 9 are permitted,
- in base 16 the characters 0 through 9 and the letters a, b, c, d, e and f, in lower or upper case, are allowed,
- if the string starts with Ox the number is assumed to be hexadecimal (base 16).

### **14)The random()**

This function in JavaScript is a built-in method that generates numbers between 0 and 1, appearing to be random. However, these numbers are not truly random because they are generated using a specific algorithm known as a pseudorandom number generator.

This algorithm is seeded from the current time, meaning that the numbers produced can vary each time you run your program but are not truly random.

Example:

```
let randomNumber1 = Math.random();
```

```
// Generates a random number between 0 and 1
console.log(randomNumber1);
```

### 15) round (value)

Returns the result of rounding its argument to the nearest integer.

```
let rounded1 = Math.round(3.4); // Rounds 3.4 to 3
console.log(rounded1);
let rounded2 = Math.round(3.6); // Rounds 3.6 to 4
console.log(rounded2);
```

### 16) sqrt (value)

Returns the square root of the value.

```
let squareRoot = Math.sqrt(9); // Returns the square root of 9, which is 3
console.log(squareRoot);
```

**Table 5.1** JavaScript Numerical Constants

| Constant     | Description                                                          |
|--------------|----------------------------------------------------------------------|
| Math.E       | Approximately 2.718. Euler's Constant, base of natural logarithms.   |
| Math.LN2     | Log of 2.                                                            |
| Math.LN10    | Log of 10.                                                           |
| Math.PI      | 3.14159. The ratio of the circumference of a circle to its diameter. |
| Math.SQRT1_2 | The square root of 1/2.                                              |
| Math.SQRT_2  | The square root of 2.                                                |

## STATEMENTS

In programming, statements are essential units that compose programs. They serve as instructions for manipulating data. A statement, on its own, typically doesn't achieve much; it needs to be part of a larger group of statements to create meaningful functionality. Programs consist of both data and code, where data represents the information being manipulated and code comprises the instructions for manipulating that data.

JavaScript statements are "instructions" to be "executed" by the web browser. Statements in javascript comprises of Values, Operators, Keywords, Expressions, and Comments. They execute sequentially.

## **IF-Else STATEMENT**

The if...else statement in JavaScript is used to make decisions based on the evaluation of a condition. If the condition evaluates to true, the code inside the if block executes. If the condition evaluates to false, the code inside the else block (if present) executes.

Here's the syntax:

```
if (condition) {  
    // code to execute if condition is true  
} else {  
    // code to execute if condition is false  
}
```

Example:

```
var num = 10;  
if (num > 0) {  
    console.log("Number is positive");  
} else {  
    console.log("Number is non-positive");  
}
```

## **SWITCH STATEMENT**

The switch statement in JavaScript provides a way to execute different blocks of code based on different conditions. It's often used as an alternative to long chains of if...else if...else statements when you need to compare a single value against multiple possible values.

Here's the syntax:

```
switch(expression) {  
    case label1:
```

```
// code to be executed if expression matches value1
break;
case label2:
    // code to be executed if expression matches value2
    break;
default:
    // code to be executed if expression doesn't match any case
}
```

- expression: The value to be evaluated in each case.
- case Label: A value or condition to compare against the expression.
- break: Keyword to terminate the switch statement. If omitted, the code will continue to execute into the next case block. This is called "fall-through."
- default: Optional. If no case matches the expression, the code inside the default block is executed.

Example:

```
var day = "Wednesday";
var message;
switch(day) {
    case "Monday":
        message = "It's Monday, start of the week.";
        break;
    case "Tuesday":
        message = "It's Tuesday, getting through the week.";
        break;
    case "Wednesday":
        message = "It's Wednesday, halfway there!";
        break;
    default:
        message = "It's another day of the week.";
}
console.log(message); // Outputs: "It's Wednesday, halfway there!"
```

If the value of day had been something other than "Monday", "Tuesday", or "Wednesday", the default case would have been executed.

## Decision making and Looping

In looping, a sequence of statements is executed until some conditions for termination of the loop are satisfied.

The following are the Looping Statements

1. While Statements
2. For Statements

## For Loop

The for loop in JavaScript is used to execute a block of code repeatedly for a specified number of times. It consists of three optional parts: initialization, condition, and iteration, separated by semicolons (;).

Here's the syntax:

```
for (initialization; condition; iteration)
{
    // code to be executed
}
```

Initialization: Executes before the loop starts and is typically used to initialize a counter variable.

Condition: Checked before each iteration. If it evaluates to true, the loop continues; if false, the loop stops.

Iteration: Executes after each iteration of the loop. It's usually used to update or increment the counter variable.

Example:

```
<html>
<head><title>For Loops</title></head>
<body>
<script language="javascript">
document.writeln("<h1>A For Loop</h1>");
```



```

for(var count = 0; count < 12; count++) {
  // repeated statements go here
  document.write("<p>The counter is " + count);
  document.writeln("</p>");
}
document.close();
</script>
</body>
</html>

```

The loop runs for 12 iterations (count starting from 0 and stopping when it reaches 11), displaying the current value of the count variable in each iteration within paragraph tags. Finally, document.close(); closes the document for writing.

## While Loop

In JavaScript, a while loop is a control flow statement that repeatedly executes a block of code as long as a specified condition evaluates to true.

Syntax:

```
while (condition)
```

```
{ // code to be executed while the condition is true }
```

- condition is a Boolean expression that is evaluated before each iteration of the loop. If the condition evaluates to true, the loop continues.

If it evaluates to false, the loop terminates.

Example:

```

var count = 0;
while (count < 5) {
  console.log("Count is: " + count);
  count++;
}

```

## **eval()**

The eval() method evaluates or executes an argument. If the argument is an expression, eval() evaluates the expression. If the argument is one or more JavaScript statements, eval() executes the statements.

### **Syntax:**

eval(string);

### **Example:**

1. Input : eval(new String('2 + 2'));

Output: returns a String object containing "2 + 2"

2. Input : eval(new String('4 + 4'));

Output: returns a String object containing "4 + 4"

## **Break**

JavaScript break statement is used to terminate the execution of the loop or the switch statement when the condition is true.

- In a switch, code breaks out and the execution of code is stopped.
- In a loop, it breaks out to the loop but the code after the loop is executed.

### **Syntax:**

break;

### **Example:**

```
const fruit = "Mango";
switch (fruit)
{
    case "Apple":
        console.log("Apple is healthy for our body");
        break;
    case "Mango":
        console.log("Mango is a National fruit of India");
        break;
    default:
        console.log("I don't like fruits.");
}
```

## Continue

The continue statement is used to skip the current iteration of a loop and continue with the next iteration.

It skips the rest of the code inside the loop's body for the current iteration and moves on to the next iteration.

Example:

```
for (var i = 0; i < 5; i++) {  
  if (i === 2) {  
    continue; // Skip the rest of the loop body for i = 2  
  }  
  console.log(i);  
}
```

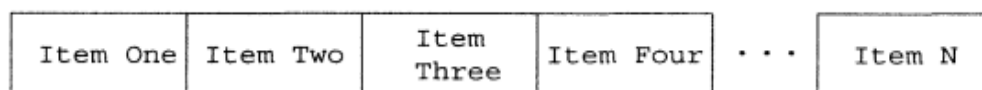
In this example, the loop prints numbers from 0 to 4, but it skips printing 2 because the continue statement is encountered when i is equal to 2.

## ARRAYS

An array is an ordered set of data elements which can be accessed through a single variable name.

We can access the values by referring to an index number; Index starts from 0 to array.length - 1.

In many programming languages array are contiguous area of memory.



**Figure 5.4** The Structure of an Array

### Basic Array Functions

The basic operations that are performed on arrays are: creation, addition of elements, accessing individual elements, removing elements.

#### I. Creating Arrays

##### 1) By Array literal

```
var days = ["Monday", "Tuesday", "Wednesday", "Thursday"];
```

2) By creating instance of array directly

```
var days = new Array ("Monday", "Tuesday", "Wednesday", "Thursday");
```

3)By using array constructor

```
var days = new Array(2);
```

```
days[0] = "monday";
```

```
days[1]="tuesday";
```

```
days[2]="wednesday";
```

JavaScript arrays can hold mixed data types as the following examples show:

```
var data = ["Monday", "Tuesday", 34, 76.34, "Wednesday"];
```

```
var data = new Array("Monday", 34, 76.34, "Wednesday");
```

## II. Adding Elements to an Array

Array elements are accessed by their index. The index denotes the position of the element in the array and, as in for loops, these start from 0. Adding an element uses the square bracket

```
var days = new Array(2);
```

```
var days = ["monday", "tuesday", "wednesday" ];
```

```
days[5] = "Thursday";
```

```
days [23] = 48;
```

### Output:

```
[ 'Monday', 'Tuesday', 'Wednesday', empty ,empty 'Thursday', <17 empty items>, 48]
```

The code creates an array of four elements in line one. A new element is added at position 5 in line two. At line three an element is added to position 23. To do this the array is first expanded so that it is long enough and then the new element is added.

## III. Accessing Array Members

The elements in the array are accessed through their index. The same access method is used to find elements and to change their value.

**Length:**

In general, accessing array elements involves utilizing their index within the array. Each element in an array is assigned a unique index, starting from 0 and ending at length - 1, where "length" represents the total number of elements stored in the array.

**Example:**

```
<script language="javascript">
document.writeln("<h1>Looping Through an Array</h1>");
document.write (" <p>")
var data = ["Monday", "Tuesday", 34, 76.34, "Wednesday"];
var len = data.length;
for (var count = 0; count < len; count++)
{
    document.write(data[count] + ", ");
}
document.writeln("</p>");
document.close ();
</script>
```

**IV. Searching an Array**

Searching an array in JavaScript involves iterating through each element of the array and comparing it with the value being searched for.

This is typically done using loops like for or while. If a match is found, specific actions can be taken, such as printing a message or returning the index of the found element.

**Example:**

```
var data = ["Monday", "Tuesday", 34, 76.34, "Wednesday"];
var len = data.length;
for (var count = 0; count < len; count++)
{
    if (data [count] == "Tuesday" )
    {
        document .write (data [count] + ", ");
    }
}
```

```

        break ;
    }
}

```

## V. Removing Array Members

Removing elements from an array in JavaScript involves manually iterating through each element and deciding whether to keep or discard it. This process typically requires creating a new array to hold the retained elements while excluding those marked for removal.

Removing an element from the array is slightly more complicated, although it is still based around a loop. The basic algorithm for the removal of an array element is:

1. create an empty, temporary array,
2. initialize separate counters for each array,
3. read the next array element,
4. compare the element with the target for removal,
5. if the two elements do not match, copy the element from the original array into the temporary array,
6. if the elements do match,
  - increment the counter on the original array,
  - copy all remaining elements from the original array into the temporary one,
7. otherwise, if the original array has any remaining elements, return to the third step.

What's happening in the algorithm is that we are copying all of the data from our array into a temporary data structure except for the element which we want to remove.

### Example:

```

<script language="javascript">
document.writeln("<h1>Removing an Array Element</h1>");
var data = ["Monday", "Tuesday", 34, 76.34, "Wednesday"]
// Show the original array
document.write("<p>");
var len = data.length;
for(var count = 0; count < len; count++) {

```

```

document.write(data[count] + ", ");
}
document.writeln ( "</p>" );
// Ask the user what to remove
var rem = prompt ( "Which item shall I remove?", "");
var tmp = new Array (data. length - 1) ;
var count2 = 0;
// This loop searches for, and removes a single item
for(var count = 0; count < len; count++)
{
    if (data [count] == rem)
    {do nothing} else
    {
        tmp[count2] = data [count] ;
        count2++;
    }
}
data = tmp ;
// Show the new Array
document.write ("" ) ;
var len = data.length;
for (var count = 0; count < len; count++)
{
    document.write (data [count] + ", ");
}
document.writeln (" " ) ;
document.close ( ) ;

```

## Object-based Array Functions

In JavaScript, arrays can indeed act like objects. This means that in addition to storing indexed values, arrays can have properties and methods just like objects. This duality allows arrays to possess both the characteristics of traditional arrays and objects.

Object-based array functions in JavaScript are methods that enable operations such as iteration, transformation, filtering, and aggregation on arrays. They provide concise and expressive ways to manipulate array elements, such as mapping each element to a new value, filtering elements based on specific criteria, or calculating aggregate values. These functions are essential for handling data in JavaScript applications, allowing developers to perform complex operations efficiently on array data structures.

### **Syntax:**

arrayname. function (parameter 1, parameter2) ;

#### **1)concat (array2 [, array 3 [, arrayN] ] )**

A list of arrays is concatenated onto the end of the array and a new array returned. The original arrays are all unaltered by this process.

#### **Example:**

```
var array1 = [1, 2, 3];  
var array2 = [4, 5, 6];  
var newArray = array1.concat(array2);  
console.log(newArray);           // Output: [1, 2, 3, 4, 5, 6]
```

#### **2) join(string)**

The join() function in JavaScript is a method used to concatenate all the elements of an array into a single string. It iterates through the elements of the array and concatenates them together, separated by a specified separator string.

#### **Example:**

```
var fruits = ["Apple", "Banana", "Orange"];  
var fruitString = fruits.join(", ");  
console.log(fruitString);        // Output: "Apple, Banana, Orange"
```

#### **3) pop()**

This function removes the last element from the array and in doing so reduces the number of elements in the array by one.

#### **Example:**

```
var numbers = [1, 2, 3, 4, 5];  
var lastNumber = numbers.pop();
```



```
console.log(numbers);           // Output: [1, 2, 3, 4]
```

#### 4) push(element1[, element2[, elementN]])

Adds a list of items onto the end of the array. The items are separated using commas in the parameter list.

**Example:**

```
var numbers = [1, 2, 3];  
numbers.push(4, 5);  
console.log(numbers);       // Output: [1, 2, 3, 4, 5]
```

#### 5) reverse()

As the name suggests, this function swaps all of the elements in the array so that which was first is last, and vice versa.

**Example:**

```
var array = [1, 2, 3, 4, 5];  
array.reverse();  
console.log(array);         // Output: [5, 4, 3, 2, 1]
```

#### 6) The sort()

This method in JavaScript arranges the elements of an array in place and returns the sorted array. By default, it sorts the elements alphabetically as strings.

**Example:**

```
var fruits = ["Banana", "Orange", "Apple"];  
fruits.sort();  
console.log(fruits);        // Output: ["Apple", "Banana", "Orange"]
```

#### 7) The slice()

This method can be used to create a copy of an array or return a portion of an array. It is important to note that the slice() method does not alter the original array but instead creates a shallow copy.

**Example:**

```
var fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];  
var citrus = fruits.slice(1, 3);  
console.log(citrus);        // Output: ["Banana", "Orange"]
```

## 8) splice()

This method is an inbuilt method in JavaScript that is used to change the contents of an array by removing or replacing existing elements and/or adding new elements.

This function is particularly useful for tasks that involve modifying arrays, such as reordering elements, deleting elements, or inserting elements into specific positions within the array.

The splice function has two compulsory parameters and an unlimited number of optional ones.

- The first parameter indicates the position in the array at which the new elements will start.
- The second parameter indicates how many elements will be deleted from the original array. If you don't want to delete elements then set this to 0.

### Example:

```
var numbers = [1, 2, 3, 4, 5];  
// Remove 2 elements starting from index 1, and insert 'a', 'b'  
numbers.splice(1, 2, 'a', 'b');  
console.log(numbers); // Output: [1, 'a', 'b', 4, 5]
```

## 9) shift()

Removes the first element of the array and in so doing shortens its length by one.

### Example:

```
var colors = ["Red", "Green", "Blue"];  
var firstColor = colors.shift();  
console.log(firstColor); // Output: "Red"  
console.log(colors);    // Output: ["Green", "Blue"]
```

## 10) unshift (element1 [, element2 [, elementN] ]

JavaScript Array unshift() Method is used to add one or more elements to the beginning of the given array. This function increases the length of the existing array by the number of elements added to the array.

### Example:

```
var fruits = ["Orange", "Apple"];  
fruits.unshift("Banana", "Mango");  
console.log(fruits); // Output: ["Banana", "Mango", "Orange", "Apple"]
```

# FUNCTIONS

In JavaScript, functions are essential for organizing code and performing specific tasks. They consist of a block of code that can be reused throughout the program. Functions can be declared using the function keyword, followed by the function name and parameters (if any), enclosed in parentheses, and the function body enclosed in curly braces. The function name should be meaningful and adhere to the same naming rules as variables.

## Defining Functions

```
function Function_name(parameters)
```

Functions are defined using the "function" keyword. The function name can be any combination of digits, letters, and underscore but cannot contain a space.

A function is a block of code and so has to have its curly brackets:

### **Syntax:**

```
function Function_name(parameters)
{
    //code goes here
}
```

### **Example:**

```
function sayHello()
{
    alert("Hello World!");
}
```

## **Function Invocation**

Function invocation refers to the act of calling or executing a function in JavaScript. Functions can be invoked by using their name followed by parentheses containing any arguments the function expects.

Example:

```
<html>
<head> <title>About You</title></head>
<body onload="about_you('Chris', 34, 8)">
  <script>
    // Function declaration
    function about_you(name, age, shoesize) {
```

```

        // Function definition: Generates HTML content based on provided
information
        document.write("<h1>All About You</h1>");
        document.write("<p><strong>Your Name is:</strong> " + name);
        document.write("<p><strong>You Are:</strong> " + age + " Years Old");
        document.writeln("<p><strong>Your Shoe Size is:</strong> " + shoesize);
        document.close();
    }
</script>
</body>
</html>

```

## Parameter Passing

Function parameters are the name listed in the function definition. Function arguments are the real values received by the function when it is invoked.

Inside the function the arguments behave as local variables.

When a function receives a value as a parameter, that value is given a name and can be accessed using that name by the function. The names of parameters are taken from the function definition and are applied in the order in which parameters are passed in.

Parameters in JavaScript functions are essential for creating flexible, customizable, and reusable code. They enable functions to accept input values, customize behaviour, promote code reuse, encapsulate logic, and ensure local scoping, contributing to the overall maintainability and efficiency of JavaScript programs.

### Example:

```

<html>
<head><title>About You</title></head>
<body onload="about_you('Chris', 34, 8)">
<script>
function about_you(name, age, shoesize)
// Function declaration with parameters: name, age, shoesize
{
    document.write("<h1>All About You</h1>");
    document.write("<p><strong>Your Name is:</strong> " + name + "</p>");
    document.write("<p><strong>You Are:</strong> " + age + " Years Old</p>");

```

```
    document.write("<p><strong>Your Shoe Size is:</strong> " + shoesize + "</p>");
}
</script>
</body>
</html>
```

The *about\_you* function is declared with three parameters: name, age, and shoesize. When the page loads (onload event), the function about\_you is invoked with specific values passed as arguments: 'Chris' for name, 34 for age, and 8 for shoesize.

Inside the function:

The values passed as parameters (name, age, and shoesize) are concatenated with HTML strings to form the content displayed on the page.

In JavaScript parameters are passed as arrays. Every function has two properties that can be used to find information about the parameters:

### **function.arguments**

This is the array of parameters that have been passed.

### **function.arguments.length**

This is the number of parameters that have been passed into the function. You could easily use this to check that all parameters have been sent and to issue a warning if they haven't.

### **Example:**

```
function sum()
{
    console.log("Number of arguments:", sum.arguments.length);
    console.log("Arguments passed:", sum.arguments);
    let total = 0;
    for (let i = 0; i < sum.arguments.length; i++)
    {
        total += sum.arguments[i];
    }
    return total;
}
console.log("Sum:", sum(1, 2, 3));
```

OUTPUT:

Number of arguments: 3  
Arguments passed: [1, 2, 3]  
Sum: 6

## Comments

Comments are pieces of text which can be used by programmers as they read through the source but which are ignored by the interpreter or compiler.

In JavaScript, there are two main types of comments:

**1)Single-Line Comments:** Single-line comments start with `//` and continue until the end of the line. They are used to add comments or annotations on a single line of code.

Example:

```
// This is a single-line comment  
var x = 10; // This comment is at the end of a line of code
```

**2)Multi-Line Comments:** Multi-line comments start with `/*` and end with `*/`. They can span multiple lines and are often used for longer comments or comment blocks.

Example:

```
/* This is a multi-line comment  
and is often used for longer comments or explanations */
```

## Returning Values

The return statement in JavaScript is used to end the execution of a function and specifies the value to be returned by that function. It can be used to return a specific value, variable, or expression to the caller. If no value is provided, the function returns undefined by default.

Functions return values to provide useful information after completing their tasks, aiding in code organization and versatility. The returned values enable seamless integration with other parts of the program, facilitating further processing.

Additionally, functions can signal errors or exceptional conditions through special return values or messages.

## Scoping Rules

Variable scope in JavaScript refers to the visibility and accessibility of variables within different parts of a JavaScript program. The scope of a variable determines where in the code that variable can be accessed and modified.

In JavaScript variables can be either local or global.

- **Global**

Global scoping means that a variable is available to all parts of the program. Such variables are declared outside of any function and are usually used to hold static data that you won't alter once it's been created.

- **Local**

Local variables are declared inside a function. They can only be used by that function. If you want the value associated with a local variable to be available to other functions then you must pass it as a parameter.

### Example:

```
var globalVariable = 10;
function myFunction()
{
    var localVariable = 20;
    console.log("Global variable:", globalVariable);
    console.log("Local variable:", localVariable);
}
console.log("Global variable:", globalVariable);
// Trying to access localVariable here will result in an error
console.log("Local variable (outside function):", localVariable);
myFunction();
```

## Objects

JavaScript does implement object-oriented (OO) features, but it has its own unique way of doing things, especially when it comes to user-defined objects.

A JavaScript object is an entity having state and behavior (properties and method).

**For example:** car, pen, bike, chair, glass, keyboard, monitor etc.

JavaScript is an object-based language. Everything is an object in JavaScript.

- **new**

The keyword **new** is used to create objects. It allocates memory and storage for them and sets all variables that can be set at this stage.

**new** calls a function which has the same name as the type of object that is being created. This function is called the constructor.

- **This**

In JavaScript, when referring to a variable that belongs to an object, we use the keyword **this** followed by a dot and then the variable name. This helps distinguish between global variables and object properties that might have the same name.

- **.(dot)**

The dot operator (.) in JavaScript is used to access properties and methods of objects. When used with objects, the dot operator accesses the property or method of the object to the left of the dot.

**Example:**

```
<script>
function emp(id,name,salary)
{
    this.id=id;
    this.name=name;
    this.salary=salary;
}
function emp(id, name, salary) {
    // Within the function, "this" refers to the newly created object
    // Assign the id, name, and salary passed as arguments to the properties of the object
    this.id = id;
    this.name = name;
    this.salary = salary;
}
// Create a new instance of the "emp" object using the "new" keyword
var e = new emp(02, "Arjun", 20000);
// Access object properties using the dot operator
document.write(e.id + " " + e.name + " " + e.salary);
    e = new emp(02,"Arjun",20000);
    doc.write(e.id,+” “+e.name+””+e.salary)
</script>
```



## REGULAR EXPRESSIONS

A regular expression is a JavaScript object. A regular expression is a sequence of characters that forms a search pattern. Regular expressions can be used to perform all types of text search and text replacement operations.

A static regular expression is created as follows:

Syntax:

```
var pattern = /pattern/attributes
```

Example: `var regex = /fish|fowl/;`

Dynamic patterns are created using the `new` keyword to create an instance of the `RegExp` class:

Syntax:

```
var pattern = new RegExp(pattern,attributes);
```

Example:

```
regex = new RegExp("fish|fowl");
```

**Table 6.1** JavaScript Regular Expression Grammar

| Token               | Description                                                                                                                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>^</code>      | Match at the start of the input string.                                                                                                                                                                                            |
| <code>\$</code>     | Match at the end of the input string.                                                                                                                                                                                              |
| <code>*</code>      | Match 0 or more times.                                                                                                                                                                                                             |
| <code>+</code>      | Match 1 or more times.                                                                                                                                                                                                             |
| <code>?</code>      | Match 0 or 1 time.                                                                                                                                                                                                                 |
| <code>a b</code>    | Match a or b.                                                                                                                                                                                                                      |
| <code>{n}</code>    | Match the string n times.                                                                                                                                                                                                          |
| <code>\d</code>     | Match a digit.                                                                                                                                                                                                                     |
| <code>\D</code>     | Match anything except for digits.                                                                                                                                                                                                  |
| <code>\w</code>     | Match any alphanumeric character or the underscore.                                                                                                                                                                                |
| <code>\W</code>     | Match anything except alphanumeric characters or underscores.                                                                                                                                                                      |
| <code>\s</code>     | Match a whitespace character.                                                                                                                                                                                                      |
| <code>\S</code>     | Match anything except for whitespace characters.                                                                                                                                                                                   |
| <code>[...]</code>  | Creates a set of characters, one of which must match if the operation is to be successful. If you need to specify a range of characters then separate the first and last with a hyphen: <code>[0-9]</code> or <code>[D-G]</code> . |
| <code>[^...]</code> | Creates a set of characters which must not match. If any character in the set matches then the operation has failed. This fails if any lowercase letter from d to q is matched: <code>[^d-q]</code> .                              |

## Functions

Regular expressions are manipulated using functions which belong to either the `RegExp` or `String` classes.

### I. Class `RegExp` Functions

### 1. exec (string)

Executes a search for a matching pattern in its parameter string. Returns an array holding the result(s) of the operation.

### 2. test(string)

Searches for a match in its parameter string. Returns true if a match is found, otherwise returns false.

## II. Class string Functions

### 1.match (pattern)

Searches for a matching pattern. Returns an array holding the results, or null if no match is found.

### 2. replace (pattern1, pattern2)

Searches for pattern1. If the search is successful pattern1 is replaced with pattern2.

### 3. search (pattern)

Searches for a pattern in the string. If the match is successful, the index, offset, of the start of the match is returned. If the search fails, the function returns -1.

### 4. split (pattern)

Splits the string into parts based upon the pattern, or regular expression, which is supplied as a parameter.

## Flags

The default behavior for regular expressions is to work only on single lines of data, to stop as soon as a match is successful and to use the pattern exactly as written.

The behavior of RegExp objects can be modified using three flags.

“i” - Performs searches which ignore the case of the pattern and the input string.

“m” - Allows searching of data which spans several input lines.

“g” - Rather than stopping when the match is successful, this forces global matching across all of the input.

The flags are applied either directly into the regular expression:

```
regex = /fish|fowl/ig;
```

or

As an additional parameter to the object constructor:

```
regex = new RegExp("fish|fowl", "ig");
```

## Exceptional Handling

In programming, exception handling is a process or method used for handling the abnormal statements in the code and executing them. It also enables to handle the flow control of the code/program. For handling the code, various handlers are used that process the exception and execute the code.

An exception in object-based programming is an object, created dynamically at run-time, which encapsulates an error and some information about it .

The great thing about exceptions is that you can define your own exception classes to include exactly what you need to handle the problem successfully.

Example:

```
<script>
  var a=4;
  var b=0;
  -----
  c=a/b;
</script>
```

Using exceptions needs two new pieces of the JavaScript language.

### **throw**

An exception is an object. It's created using the standard new method. Once the exception object exists, you need to do something with it. What you do is throw the exception, that is you pass it back up the call stack until there's a piece of code which can handle it.

The syntax of the throw is:

```
do something
if an error happens {
  create a new exception object
  throw the exception
}
```

### **try...catch**

Where you have a block of code which might cause the creation of an exception, you need to program some code to handle the exception if it should happen.

The try. . . catch mechanism is found in many programming languages not just

JavaScript. The idea here is that your program is going to try to execute a block of statements. If an exception is thrown by any of those statements, execution of the whole block ceases and the program looks for a catch statement to handle the exception, try. . . catch blocks take this form:

```
try {  
statement one  
statement two  
statement three  
} catch exception {  
handle the exception  
}
```

If statement one throws an exception, statement two and statement three will not be executed. Instead the program will move straight to the catch.

**Example:**

```
// Define a function to divide two numbers
```

```
function divide(num1, num2) {  
  if (num2 === 0) {  
    // Throw an exception if the divisor is zero  
    throw "Division by zero is not allowed";  
  }  
  return num1 / num2;  
}
```

```
// Try block to handle exceptions
```

```
try {  
  // Attempt to divide 10 by 0  
  var result = divide(10, 0);  
  console.log("Result:", result); // This line won't execute if an exception is thrown  
} catch (error) {  
  // Catch block to handle exceptions  
  console.error("Error:", error); // Log the error message  
}
```

## BUILTIN OBJECTS

In JavaScript, there are numerous built-in objects provided by the browser environment, which are essential for scripting.

### **The Document Object OR Document Object Model(DOM)**

A document is a Web page that is being either displayed or created. The document has a number of properties that can be accessed by JavaScript programs and used to manipulate the content of the page. Some of these properties can be used to create HTML pages from within JavaScript while others may be used to change the operation of the current page.

1)write() and writeln() Methods: These methods are used to dynamically create HTML content within JavaScript. They allow you to write HTML code directly into the document.

For example:

```
document.write("<body>");  
document.write("<h1>A test</h1>");  
document.write("<form>");
```

2)bgcolor and fgcolor Properties: These properties set the background and foreground colors of the document, respectively. They accept hexadecimal values or color names:

```
document.bgcolor = "#e302334";  
document.fgcolor = "coral";
```

3)anchors Property: An array containing the names of all anchors (named points) within the HTML document. Anchors are created using <a name="..."> tags:

```
document.anchors[0];
```

4)links Property: An array holding information about all the links on the web page, in the order they appear.

5)forms Property: An array containing all HTML forms on the page. It allows access to individual form elements within each form.

6)layers Property: If the document is composed of layers (e.g., in older HTML design using <layer> tags), this property holds an array of layer objects. Layers have their own methods and properties.

7)close() Method: This method signals that the document writing is complete. It's essential to call close() after using write() or writeln(). If not called, the browser might continue waiting for more data.

## **Window objects**

The browser window is a mutable object that can be addressed by JavaScript code. The Window object provides access to various properties and methods that enable interaction with the browser environment, including manipulating the document, handling events, managing timers, displaying dialog boxes, and more.

some of the properties and methods that are available from Window objects,

1)open("URL", "name")

This opens a new window which contains the document specified by URL. The window is given an identifying name so that it can be manipulated individually.

2)close()

This shuts the current window.

3) When positioning elements on web pages, especially for dynamic content, it's helpful to be able to place them regardless of the screen resolution or window size. Two key properties for this are width and height.

- width: Sets the width of an element, like an image or window, in pixels.
- height: Sets the height of an element in pixels.

|                               |                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------|
| <b>full screen=yes no 1 0</b> | <b>Whether or not to display the browser in full-screen mode. Default is no</b> |
| <b>height=pixels</b>          | The height of the window. Min. value is 100                                     |
| <b>location=yes no 1 0</b>    | Whether or not to display the address field.                                    |
| <b>menubar=yes no 1 0</b>     | Whether or not to display the menu bar                                          |
| <b>resizable=yes no 1 0</b>   | Whether or not the window is resizable.                                         |
| <b>scrollbars=yes no 1 0</b>  | Whether or not to display scroll bars.                                          |
| <b>status=yes no 1 0</b>      | Whether or not to add a status bar                                              |
| <b>titlebar=yes no 1 0</b>    | Whether or not to display the title bar.                                        |
| <b>toolbar=yes no 1 0</b>     | Whether or not to display the browser toolbar                                   |
| <b>top=pixels</b>             | The top position of the window. Negative values not allowed                     |
| <b>width=pixels</b>           | The width of the window. Min. value is 100                                      |

## Frame Objects

In JavaScript, the frames property of the HTML DOM Window object is utilized to access <iframe> elements within the current window. It returns these frame elements as an array object, allowing for easy manipulation and interaction.

These frame objects are essentially children of the window object, automatically generated by the browser. They play a crucial role in managing and controlling the loading and accessing of frames within the web page.

## Form Object

JavaScript's Form Object empowers developers to manipulate and interact with HTML forms dynamically.

This essential feature enables two crucial functionalities:

- a) data validation before submission
- b) dynamic form creation.

By utilizing JavaScript's Form Object, developers can create dynamic, responsive forms that enhance user experience and ensure data integrity. This pivotal aspect of

JavaScript empowers developers to craft interactive web applications with robust form functionalities.

These events can be used to trigger validation routines:

- `onclick="method"`

This can be applied to all form elements. The event is triggered when the user clicks on that element. It is not triggered if you try to force events through the `click()` method.

- `onSubmit="method"`

This event can only be triggered by the form itself and occurs when a form is submitted.

- `onReset="method"`

Like the previous one this is a form-only event and is (obviously) triggered when a form is reset by the user.

## Browser Object

No two browser models will process your JavaScript in the same way. It's important that you try to find out which browser is being used to view your page. You can then make a choice for your visitors:

- exclude browsers that are unable to use your code;
- redirect them to a non-scripted version of your site;
- present scripts that are tailored to suit each browser.

The browser object is called as navigator object

- `navigator.appCodeName`

The internal name for the browser. For both major products this is Mozilla, which was the name of the original Netscape code source.

- `navigator.appName`

This is the public name of the browser - navigator or Internet explorer for the big two.

- `navigator.appVersion`

The version number, platform on which the browser is running, and (for Internet Explorer) the version of Navigator with which it is compatible.

- `navigator.userAgent`

The strings `appCodeName` and `appVersion` concatenated together.

- `navigator.plugins`

An array containing details of all installed plug-ins.



➤ `navigator .mimeTypes`

An array of all supported MIME types - useful if you need to make sure that the browser can handle your data.

## **Browser Sniffing**

Browser sniffing is like peeking at a visitor's web browser to see what they're using. It looks at something called the user agent string, which is like a browser's ID card. By checking this ID card, websites can make choices, like showing different features or designs depending on the browser.

## **EVENTS**

JavaScript is an event-driven system.

- JavaScript Events are actions or occurrences that happen in the browser. They can be triggered by various user interactions or by the browser itself.
- Common events include mouse clicks, keyboard presses, page loads, and form submissions. Event handlers are JavaScript functions that respond to these events, allowing developers to create interactive web applications.

The concept of event handlers revolves around associating JavaScript functions with HTML elements or browser events. When the specified event occurs, the associated event handler function is automatically invoked, allowing you to perform actions or execute code in response to that event.

Event handlers can be added to HTML elements directly within the HTML code using attributes like `onclick`, `onmouseover`, `onsubmit`, etc. Alternatively, they can be attached dynamically to elements using JavaScript code.

**Table 6.3** JavaScript Events

| Event     | Handler     | Description                                                                                                   |
|-----------|-------------|---------------------------------------------------------------------------------------------------------------|
| blur      | onBlur      | The input focus is moved from the object, usually when moving from a field of a form or from the form itself. |
| change    | onChange    | The value of a field in a form has been changed by the user entering or deleting data.                        |
| click     | onClick     | The mouse is clicked over an element of a page.                                                               |
| dblclick  | onDbClick   | A form element or a link is clicked twice in rapid succession.                                                |
| dragdrop  | onDragDrop  | A system file is dragged with a mouse and dropped onto the browser.                                           |
| focus     | onFocus     | Input focus is given to an element. The reverse of blur.                                                      |
| keydown   | onKeyDown   | A key is depressed but not released.                                                                          |
| keypress  | onKeyPress  | A key is pressed.                                                                                             |
| keyup     | onKeyUp     | A depressed key is released.                                                                                  |
| load      | onLoad      | The page is loaded by the browser.                                                                            |
| mousedown | onMouseDown | A mouse button is pressed.                                                                                    |
| mousemove | onMouseMove | The mouse, and hence cursor, is moved.                                                                        |
| mouseout  | onMouseOut  | The mouse pointer moves off an element.                                                                       |
| mouseover | onMouseOver | The mouse pointer is moved over an element.                                                                   |
| mouseup   | onMouseUp   | The mouse button is released.                                                                                 |
| move      | onMove      | A window is moved, maximized or restored either by the user or by a script.                                   |
| resize    | onResize    | A Window is resized by the user or by a script.                                                               |
| select    | onSelect    | A field on a form is selected by clicking the mouse or tabbing from the keyboard.                             |
| submit    | onSubmit    | A form is submitted (the submit button is clicked).                                                           |
| unload    | onUnload    | The user leaves the Web page.                                                                                 |

**Example:**

```

<html><head><title>Handling Events</title></head>
<body onLoad="ShowLoaded()" onUnload="SayGoodbye()">
<h1>Handling Events</h1>
<p>
<a href="#" onMouseOver="Mouse()">A Hyperlink</a>
</p>
<form>
<input type="button"
value="Click Me!"
onClick="Clicked()"
onMouseOver="Mouse()">
</form>
<script language="javascript">
function ShowLoaded(){

```

```
alert("The page has loaded");
}
function SayGoodbye() {
alert("Goodbye, thanks for visiting");
}
function Clicked ( ) {
alert ("You clicked the button")
}
function Mouse ( ) {
alert ("The mouse is over the link");
}
</script>
</body>
</html>
```

## **Boolean and Number Objects**

### **Boolean Object**

- JavaScript provides the Boolean and Number objects as object wrappers for boolean true/false values and numbers, respectively. These wrappers define methods and properties useful in manipulating boolean values and numbers.
- When a JavaScript program requires a boolean value, JavaScript automatically creates a Boolean object to store the value. JavaScript programmers can create Boolean objects explicitly with the statement.  
var b = new Boolean( booleanValue );
- The booleanValue specifies whether the Boolean object should contain true or false. If booleanValue is false, 0, null, NaN or an empty string (""), or if no argument is supplied, the new Boolean object contains false. Otherwise, the new Boolean object contains true.

| Method                  | Description                                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>toString()</code> | Returns the string "true" if the value of the Boolean object is true; otherwise, returns the string "false". |
| <code>valueOf()</code>  | Returns the value true if the Boolean object is true; otherwise, returns false.                              |

**Fig. 11.13** | Boolean-object methods.

## Number Objects

JavaScript automatically creates Number objects to store numeric values in a script.

You can create a Number object with the statement.

```
var n = new Number( numericValue );
```

The constructor argument `numericValue` is the number to store in the object. Although you can explicitly create Number objects, normally the JavaScript interpreter creates them as needed.

| Method or property                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>toString( radix )</code>        | Returns the string representation of the number. The optional <i>radix</i> argument (a number from 2 to 36) specifies the number's base. Radix 2 results in the <i>binary</i> representation, 8 in the <i>octal</i> representation, 10 in the <i>decimal</i> representation and 16 in the <i>hexadecimal</i> representation. See Appendix E, Number Systems, for an explanation of the binary, octal, decimal and hexadecimal number systems. |
| <code>valueOf()</code>                | Returns the numeric value.                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>Number.MAX_VALUE</code>         | The largest value that can be stored in a JavaScript program.                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>Number.MIN_VALUE</code>         | The smallest value that can be stored in a JavaScript program.                                                                                                                                                                                                                                                                                                                                                                                |
| <code>Number.NaN</code>               | <i>Not a number</i> —a value returned from an arithmetic expression that doesn't result in a number (e.g., <code>parseInt("hello")</code> cannot convert the string "hello" to a number, so <code>parseInt</code> would return <code>Number.NaN</code> .) To determine whether a value is NaN, test the result with function <code>isNaN</code> , which returns true if the value is NaN; otherwise, it returns false.                        |
| <code>Number.NEGATIVE_INFINITY</code> | A value less than <code>-Number.MAX_VALUE</code> .                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>Number.POSITIVE_INFINITY</code> | A value greater than <code>Number.MAX_VALUE</code> .                                                                                                                                                                                                                                                                                                                                                                                          |

**Fig. 11.14** | Number-object methods and properties.

## Using JSON to Represent Objects

JSON (JavaScript Object Notation)—a simple way to represent JavaScript objects as strings—is a simpler alternative to XML for passing data between the client and the server.

Each object in JSON is represented as a list of property names and values contained in curly braces, in the following format:

```
{ "propertyName1" : value1, "propertyName2": value2 }
```

Arrays are represented in JSON with square brackets in the following format:

```
[ value0, value1, value2 ]
```

Each value can be a string, a number, a JSON object, true, false or null.

```
[ { first: 'Cheryl', last: 'Black' },  
  { first: 'James', last: 'Blue' },  
  { first: 'Mike', last: 'Brown' },  
  { first: 'Meg', last: 'Gold' } ]
```

JSON provides a straightforward way to manipulate objects in JavaScript, and many other programming languages now support this format. In addition to simplifying object creation, JSON allows programs to easily extract data and efficiently transmit it across the Internet.

You can convert JSON strings into JavaScript objects with JavaScript's JSON parse function. JSON strings are easier to create and parse than XML and require fewer bytes.

For these reasons, JSON is commonly used to communicate in client/server interaction.